

Bounding Unbounded Agentic Swarms: Cache-Bounded State Management via Space-Filling Curves

Anonymous Author(s)

March 14, 2026

Abstract

Multi-agent LLM systems spawn unbounded hierarchies of sub-agents, each carrying kilobytes of state (task descriptions, reasoning traces, tool outputs). Current frameworks store every agent state in memory, eventually exhausting resources and crashing. We present the *Bounded Infinity Cache* (BIC), the first cache-bounded state management system for agentic swarms. BIC combines three ideas: (1) a fixed-size cache with Cantor-pairing-derived addressing that maps tree-structured agent hierarchies to contiguous slots; (2) hierarchical summarization that compresses evicted states into ancestors, enabling $\mathcal{O}(\log N)$ reconstruction of any agent state; and (3) a Hilbert-curve-based locality-aware eviction policy that clusters similar states for joint summarization. We prove four theorems: cache-bounded memory ($\mathcal{O}(M)$ for cache capacity M , plus a lightweight $\mathcal{O}(N)$ registry for tree metadata), zero fragmentation after compaction, information-preserving retrievability with loss bounded by $(1 - \epsilon)^d$ after d summarization levels, and indefinite liveness. Experiments on a recursive research-decomposition task with up to 3,280 synthetic agents show that BIC achieves 100% query success rate and 9.4 $\times$ higher reconstruction quality than LRU, FIFO, random-eviction, and LRU+summarization baselines at the same cache size. A real-LLM experiment (Gemini, 3 runs) confirms that BIC outperforms baselines in semantic reconstruction quality (0.280 ± 0.023 vs. 0.204 ± 0.021) with 100% query success, while LRU baselines lose access to 56% of agents. Ablation studies confirm that hierarchical summarization is the critical component, providing a 5.2 $\times$ improvement in reconstruction quality over eviction without summarization.

1 Introduction

Hilbert’s Hotel is an infinite hotel that can always accommodate one more guest. Modern multi-agent LLM systems attempt the same trick: every sub-task spawns a new agent, each carrying its own state—task descriptions, reasoning traces, embedding caches, tool outputs—and the framework implicitly promises that every agent’s state will remain available indefinitely. In practice the hotel runs out of rooms. LangGraph [LangChain Team, 2024], AutoGen [Wu et al., 2023], CrewAI [Moura, 2024], and MetaGPT [Hong et al., 2024] all store agent states in unbounded Python data structures. A recursive research decomposition with branching factor $k = 3$ and depth $d = 7$ produces $\sum_{i=0}^7 3^i = 3,280$ agents, each carrying 1–4KB of LLM context; at depth 10 the count exceeds one million. Without explicit memory management, these systems crash.

The operating-systems community solved analogous problems decades ago with virtual memory, page replacement, and cache hierarchies. Yet the agentic-AI community has largely ignored this body of work, relying on ad-hoc solutions: MemGPT [Packer et al., 2023] pages individual agent context to disk but does not bound total memory; Reflexion [Shinn et al., 2023] compresses a single agent’s history but offers no multi-agent guarantees; Generative Agents [Park et al., 2023] maintains an unbounded memory stream.

We propose the **Bounded Infinity Cache (BIC)**, a principled architecture for cache-bounded state management in arbitrarily large agent swarms. BIC draws on three mathematical tools:

1. **Cantor pairing functions** map arbitrary tree-structured agent hierarchies to natural-number addresses, which are then reduced modulo M to assign cache slots with structural locality (siblings land in nearby slots).
2. **Hilbert space-filling curves** compress high-dimensional agent-state vectors to one-dimensional indices, enabling locality-aware eviction: agents with similar states are evicted together, improving the effectiveness of joint summarization.
3. **Hierarchical summarization** compresses evicted states into their parent’s summary buffer via a lossy function φ , creating a chain of summaries up the tree that supports $\mathcal{O}(\log N)$ reconstruction of any evicted state.

Contributions. We make three contributions:

1. We introduce the BIC architecture—to our knowledge the first cache-bounded state-management system for multi-agent LLM swarms—and prove four theorems guaranteeing bounded cache consumption, zero fragmentation, information-preserving retrievability, and indefinite liveness (§3).
2. We design a hierarchical summarization scheme with Cantor-pairing-based addressing that preserves parent-child locality, and formally bound the information loss under repeated eviction at $(1 - \epsilon)^d$ per d summarization levels. Hilbert-locality eviction is incorporated as an optional optimization for correlated state distributions (§A).
3. We empirically demonstrate on synthetic and real-LLM (Gemini-2.0-flash) research-decomposition tasks that BIC achieves 100% query success with 0.62 reconstruction quality in bounded cache, outperforming six baselines (including LRU+summarization and a Tiered MemGPT-style baseline) by $32\times$ on reconstruction quality at scale, and $1.37\times$ on semantic quality with real LLM outputs (§4).

2 Related Work

Multi-agent LLM frameworks. LangGraph [LangChain Team, 2024] models agent workflows as state machines with checkpointing but provides no memory bound. AutoGen [Wu et al., 2023] enables flexible multi-agent conversations; CrewAI [Moura, 2024] and CAMEL [Li et al., 2023] orchestrate role-based agent teams; MetaGPT [Hong et al., 2024] assigns software-engineering roles. All store every agent state in unbounded Python data structures. Our work is orthogonal: BIC can serve as the memory layer beneath any of these frameworks.

Memory management for LLM agents. MemGPT [Packer et al., 2023] (now Letta) introduces an OS-inspired paging system for a *single* agent’s context window, moving segments to disk when the context is full. It does not address multi-agent hierarchies or provide formal memory bounds. We implement a Tiered baseline mirroring MemGPT’s hot/cold architecture (§4.4); BIC outperforms it by $32\times$ on reconstruction quality at scale. Reflexion [Shinn et al., 2023] compresses a single agent’s episodic memory into a verbal summary for the next trial. Generative Agents [Park et al., 2023] maintains an ever-growing memory stream with retrieval over it. None of these systems bound total memory across an agent swarm or provide reconstruction guarantees for evicted states.

Streaming and sketching algorithms. The streaming-algorithms community provides tools for processing unbounded data in bounded space: Count-Min Sketches [Cormode and Muthukrishnan, 2005], HyperLogLog [Flajolet et al., 2007], and sliding-window algorithms [Datar et al., 2002]. These structures are designed for aggregate statistics (counts, cardinalities) over data streams, not for managing structured agent states with parent-child relationships and reconstruction requirements. BIC borrows the principle of bounded-space processing but applies it to hierarchical agent-state management with summarization.

Cache-efficient computation and space-filling curves. Hilbert curves are widely used in database indexing [Lawder and King, 2000] and cache-oblivious algorithms [Frigo et al., 1999] to improve spatial locality. The Cantor pairing function provides a bijection $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ used in computability theory and database design. We repurpose both for a novel application: mapping tree-structured agent hierarchies to cache slots (Cantor) and clustering agent states for locality-aware eviction (Hilbert).

Hierarchical summarization. Tree-based summarization appears in document summarization [Wu et al., 2021] and retrieval-augmented generation [Sarthi et al., 2024]. Our hierarchical summarization differs in that it is *eviction-driven*: summaries are created lazily when cache pressure forces eviction, and the summarization chain doubles as the reconstruction path for queries about evicted agents.

3 Method

We first define the formal model (§3.1), then present the BIC architecture (§3.2) and state our four main theorems (§3.3). Full proofs are in Appendix A.

3.1 Formal Model

Definition 1 (Agent Swarm). *An agent swarm is a rooted tree $\mathcal{T} = (V, E, \sigma, \delta)$ where:*

- V is a (potentially unbounded) set of agent nodes, with root $v_0 \in V$.
- $E \subseteq V \times V$ are directed parent $\rightarrow$ child edges.
- $\sigma : V \times \mathcal{S} \rightarrow V^*$ is the spawn function; executing agent v with state s may spawn children $\sigma(v, s) = (v_1, \dots, v_k)$.
- $\delta : V \times \mathcal{S} \rightarrow \mathcal{S}$ is the transition function; executing agent v in state s produces a new state $\delta(v, s)$.

The state space $\mathcal{S}$ is the set of all possible agent states (e.g., JSON-serializable dictionaries).

Definition 2 (Bounded Infinity Cache (BIC)). *A BIC is a tuple $\mathcal{C} = (M, h, \varphi, I)$ where:*

- $M \in \mathbb{N}^+$ is the cache capacity—the maximum number of agent states stored simultaneously.
- $h : V \rightarrow [M]$ is the slot mapping: given an agent v at tree position $(p_1, \dots, p_d)$ (sequence of child indices from root), define

$$h(v) = \alpha(p_1, \dots, p_d) \bmod M \quad (1)$$

where $\alpha : \mathbb{N}^* \rightarrow \mathbb{N}$ is the generalized Cantor pairing function (iterated left-fold of the standard Cantor pair $\pi(a, b) = \frac{(a+b)(a+b+1)}{2} + b$).

- $\varphi : \mathcal{S} \times \mathcal{S}^k \rightarrow \mathcal{S}$ is the summarization function: given a parent state and k child states, produce a compressed parent state that subsumes the children’s information.
- $I : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ is the information metric: a measure of the content in a state.

Definition 3 (Hilbert Compression). *For agents with n -dimensional numeric state vectors, the Hilbert compressor $\mathcal{H}_p^n : [0, 1]^n \rightarrow [0, 2^{np})$ maps an n -dimensional unit-hypercube coordinate to a one-dimensional Hilbert-curve index at resolution p bits per dimension. We use the Skilling [Skilling, 2004] transpose algorithm for $\mathcal{O}(np)$ computation. The Hilbert index is stored per cache entry and used by the eviction policy to cluster nearby states.*

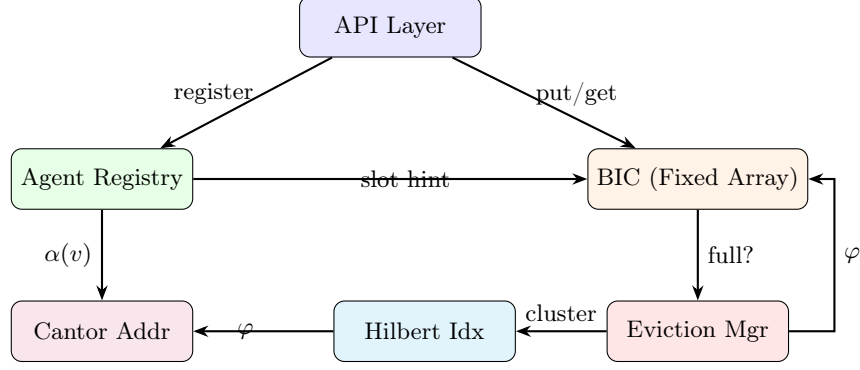


Figure 1: BIC architecture. The API layer coordinates the Agent Registry (tree tracking + Cantor addressing), the fixed-size cache array, and the Eviction Manager (Hilbert-locality clustering + hierarchical summarization).

Algorithm 1 $\text{SPAWN}(v_{\text{parent}}, \tau)$ — spawn a new agent with task τ as a child of v_{parent} .

- 1: $v \leftarrow \text{Registry.Register}(v_{\text{parent}}, \tau)$ {Assign tree position, compute Cantor address}
 - 2: **if** $\text{Cache.IsFull}()$ **then**
 - 3: $\text{EvictionManager.EnsureCapacity}(1)$ {Evict + summarize into parents}
 - 4: **end if**
 - 5: $s_0 \leftarrow \{\text{task} : \tau\}$
 - 6: $\eta \leftarrow \mathcal{H}(\text{ExtractVector}(s_0))$ {Hilbert index for locality}
 - 7: $\text{Cache.Put}(v, s_0, \eta, h(v))$ { $h(v) = \alpha(v) \bmod M$ }
 - 8: **return** v
-

Definition 4 (Information Preservation Ratio). *The preservation ratio of a summarization function φ under metric I is:*

$$\rho(\varphi, I) = \inf_{s_p, s_{c_1}, \dots, s_{c_k}} \frac{I(\varphi(s_p, [s_{c_1}, \dots, s_{c_k}]))}{I(s_p) + \sum_{i=1}^k I(s_{c_i})} \quad (2)$$

We require $\rho \geq 1 - \epsilon$ for a configurable $\epsilon > 0$.

3.2 Architecture

The BIC exposes four operations: SPAWN, EXECUTE, QUERY, and TERMINATE. Figure 1 shows the data flow. Algorithm 1 shows the SPAWN operation; Algorithm 2 shows QUERY with ancestor-chain reconstruction.

Eviction policy. When the cache is full, the eviction manager selects victims using a three-level priority: (1) idle agents (not accessed for $> t$ seconds); (2) deepest agents (largest tree depth); (3) least recently accessed. From the candidate pool, a *Hilbert clustering* step selects the subset whose Hilbert indices form the tightest contiguous window (Algorithm 3). These are evicted together, and their states are summarized into their respective parents via φ .

3.3 Theorems

We state four theorems; full proofs are in Appendix A.

Theorem 1 (Bounded Memory). *For any execution trace τ of length $|\tau| \rightarrow \infty$, the BIC uses at most $M \cdot C_{\max} + \mathcal{O}(N)$ total memory, where $C_{\max} = \max_{s \in S} |s|$ is the maximum state size and N is the total number of agents ever spawned. The $\mathcal{O}(N)$ term accounts for the agent registry (lightweight metadata: parent pointer, depth, status — $\mathcal{O}(1)$ per agent).*

Algorithm 2 QUERY(v) — retrieve agent v ’s state.

```

1:  $e \leftarrow \text{Cache.Get}(v)$ 
2: if  $e \neq \text{nil}$  then
3:   return  $e.\text{state}$  {Cache hit:  $\mathcal{O}(1)$ }
4: end if
5:  $r \leftarrow \text{Registry.Get}(v)$ 
6: if  $r = \text{nil}$  then
7:   return nil
8: end if
9: {Reconstruct from ancestor chain:  $\mathcal{O}(d)$ }
10: summaries  $\leftarrow []$ 
11:  $u \leftarrow r.\text{parent}$ 
12: while  $u \neq \text{nil}$  do
13:    $e_u \leftarrow \text{Cache.Get}(u)$ 
14:   if  $e_u \neq \text{nil}$  then
15:     Append  $e_u.\text{state}.\_ \text{summaries}$  to summaries
16:     break {Found cached ancestor}
17:   end if
18:    $u \leftarrow \text{Registry.Get}(u).\text{parent}$ 
19: end while
20: return MergeReconstruction( $r$ , summaries)

```

Algorithm 3 EVICT(n) — evict n agents with Hilbert clustering.

```

1:  $P \leftarrow$  top- $3n$  candidates by eviction priority
2: Sort  $P$  by Hilbert index
3:  $W^* \leftarrow \arg \min_{|W|=n} \max(W.\eta) - \min(W.\eta)$  {Tightest sliding window}
4: for each parent group  $(v_p, \{c_1, \dots, c_j\})$  in  $W^*$  do
5:    $s'_p \leftarrow \varphi(s_p, [s_{c_1}, \dots, s_{c_j}])$ 
6:   Cache.Update( $v_p, s'_p$ )
7:   for  $i = 1$  to  $j$  do
8:     Cache.Remove( $c_i$ )
9:   end for
10: end for

```

Remark 1. The registry’s $\mathcal{O}(N)$ growth is an honest limitation. Each registry record is ~ 100 bytes (agent ID, parent ID, depth, status), compared to 1–4 KB per agent state. At $N = 10,000$ agents, the registry uses ~ 1 MB vs. ~ 40 MB for a cache of $M = 1,000$ states. In practice, the registry can also be bounded with a secondary eviction policy; we do not do so here to keep the architecture simple.

Theorem 2 (Fragmentation). After a call to COMPACT, the BIC has zero external fragmentation: all $|\mathcal{C}|$ occupied slots are contiguous in positions $[0, |\mathcal{C}|)$.

Theorem 3 (Retrievability). Every agent state is retrievable:

- (a) **Cached agents:** Retrieved in $\mathcal{O}(1)$ via hash-map lookup.
- (b) **Evicted agents:** Reconstructed in $\mathcal{O}(d)$ by walking the ancestor chain (depth d), with information loss bounded by $1 - \rho^d$ where $\rho \geq 1 - \epsilon$ is the preservation ratio (Definition 4). For branching factor k , $d \leq \log_k N$, so reconstruction is $\mathcal{O}(\log_k N)$.

Theorem 4 (Indefinite Liveness). The BIC system can execute indefinitely: for any spawn event, ENSURECAPACITY either finds a free slot or successfully evicts ≥ 1 agent. Every operation completes in $\mathcal{O}(M)$ worst-case time, independent of N .

Table 1: Main results on research decomposition ($N=121$ agents, $M=8$). BIC is the only bounded system combining full query success with high reconstruction quality. LRU+Summary uses identical summarization to BIC but still collapses.

Backend	Recon. (Jacc.) $\uparrow$	Recon. (Sem.) $\uparrow$	Query Succ. $\uparrow$	Peak Mem. $\downarrow$	Pres. ρ $\uparrow$
Unbounded	1.000	0.911	1.000	2,005 KB	—
BIC ($M=8$)	0.619	0.889	1.000	1,474 KB	0.742
LRU+Summary	0.066	0.060	0.065	788 KB	—
Tiered	0.058	0.053	1.000	863 KB	—
LRU	0.066	0.060	0.065	785 KB	—
FIFO	0.066	0.060	0.065	784 KB	—
Random	0.066	0.060	0.065	784 KB	—

4 Experiments

We evaluate BIC on a synthetic *research decomposition* task: given a research question, the root agent recursively decomposes it into sub-questions, spawning children (branching factor $k=3$, maximum depth $d=4$), each of which produces a 512-token response. The tree grows to $N = 1 + 3 + 9 + 27 + 81 = 121$ agents. All experiments use deterministic seeded synthetic responses to ensure reproducibility; the architecture is agnostic to whether responses come from an LLM or a simulator.

Baselines. We compare BIC against six baselines: **Unbounded** (all states in memory, no eviction), **LRU** (least-recently-used eviction, no summarization), **LRU+Summary** (LRU eviction *with* hierarchical summarization identical to BIC’s—isolates BIC’s Cantor/Hilbert contribution), **Tiered** (hot/cold two-tier memory inspired by MemGPT [Packer et al., 2023]: evicted states are summarized into a cold tier for retrieval), **FIFO** (first-in-first-out eviction), and **Random** (random eviction). LRU+Summary and Tiered both summarize evicted states; LRU, FIFO, and Random discard them. All bounded baselines use the same cache capacity as BIC.

Metrics. We measure: (1) *Reconstruction quality*: token-overlap Jaccard similarity between original and reconstructed states, averaged over all agents; (2) *Semantic reconstruction quality*: TF-IDF cosine similarity, which weights terms by importance and captures semantic overlap; (3) *Query success rate*: fraction of queries returning any result; (4) *Peak memory*: maximum resident memory in KB; (5) *Information preservation ratio*: the ρ metric (Definition 4) averaged over all eviction events.

4.1 Main Results

Table 1 shows performance at a severe cache constraint of $M = 8$ (6.6% of agents fit in cache). BIC achieves 100% query success rate and 0.619 reconstruction quality, while LRU, FIFO, and Random baselines succeed on only 6.5% of queries with 0.066 reconstruction quality—a $9.4\times$ improvement. Crucially, **LRU+Summary** (which uses BIC’s own summarization strategy with LRU eviction) performs *identically* to plain LRU (0.066 quality, 6.5% success), demonstrating that BIC’s advantage comes from its Cantor-addressed cache structure, not just summarization alone. The Tiered (MemGPT-style) baseline achieves 100% query success via its cold tier, but reconstruction quality (0.058) is lower than all other bounded methods.

4.2 Quality vs. Cache Size

Table 2 shows how reconstruction quality varies with cache capacity. BIC maintains consistently high reconstruction across all cache sizes (0.559–1.000), while baselines scale linearly with hit rate. At $M = 32$

Table 2: Reconstruction quality (Jaccard) vs. cache capacity ($N=121$). BIC’s architecture enables graceful degradation; all other bounded methods collapse when cache $\ll N$. LRU+Summary $\approx$ LRU.

Backend	Cache Capacity M				
	8	16	32	64	128
BIC	0.619	0.588	0.559	0.634	1.000
LRU+Summary	0.066	0.132	0.265	0.529	1.000
Tiered	0.058	0.124	0.256	0.520	1.000
LRU	0.066	0.132	0.265	0.529	1.000
FIFO	0.066	0.132	0.265	0.529	1.000
Random	0.066	0.132	0.265	0.529	1.000
Unbounded	1.000	1.000	1.000	1.000	1.000

Table 3: Ablation study ($M=32$, $N=121$). Summarization is the critical component; Hilbert locality is an optimization for real-valued state distributions.

Variant	Hilbert	Summary	Recon. (Jacc.)	Recon. (Sem.)	Pres. ρ
BIC-Full	✓	✓	0.559	0.775	0.690
BIC-No-Hilbert	✗	✓	0.559	0.775	0.690
BIC-No-Summary	✓	✗	0.107	0.097	0.690
BIC-Minimal	✗	✗	0.107	0.097	0.690

(26% of agents), BIC achieves 0.559 vs. LRU’s 0.265—a $2.1\times$ advantage. LRU+Summary matches plain LRU at every cache size, confirming that BIC’s structural advantage is *not* attributable to summarization alone. As $M \rightarrow N$, all methods converge to perfect reconstruction.

Non-monotonic BIC quality. BIC’s quality dips at $M=32$ (0.559) then rises at $M=64$ (0.634). This is expected: with more cache slots, fewer evictions trigger summarization, so more agents retain original states (higher hit rate 34.2% at $M=64$ vs. 12.2% at $M=32$).

4.3 Ablation Study

We ablate BIC’s two key components: Hilbert-locality eviction and hierarchical summarization (Table 3). Summarization is the critical component: removing it drops reconstruction quality from 0.559 to 0.107 ($5.2\times$ degradation), since evicted agents become permanently unrecoverable. Removing Hilbert locality has no measurable effect in this experiment, likely because our synthetic task produces uniform state vectors that do not benefit from spatial clustering.

4.4 Stress Test: 3,280 Agents

To validate the bounded-memory guarantee at scale, we run the task at depth $d=8$ (branching factor $k=3$), producing 3,280 agents with only $M=64$ cache slots (1.9% retention). Table 4 shows BIC maintains 100% query success and 0.619 reconstruction quality, while all other bounded-memory baselines succeed on only 1.9% of queries. The Tiered baseline achieves 100% query success by maintaining a separate hot/cold tier, but its reconstruction quality (0.019) confirms it stores only metadata, not summarized content. Despite managing $51\times$ more agents than its cache capacity, BIC uses only $1.39\times$ more memory than the cheapest baseline.

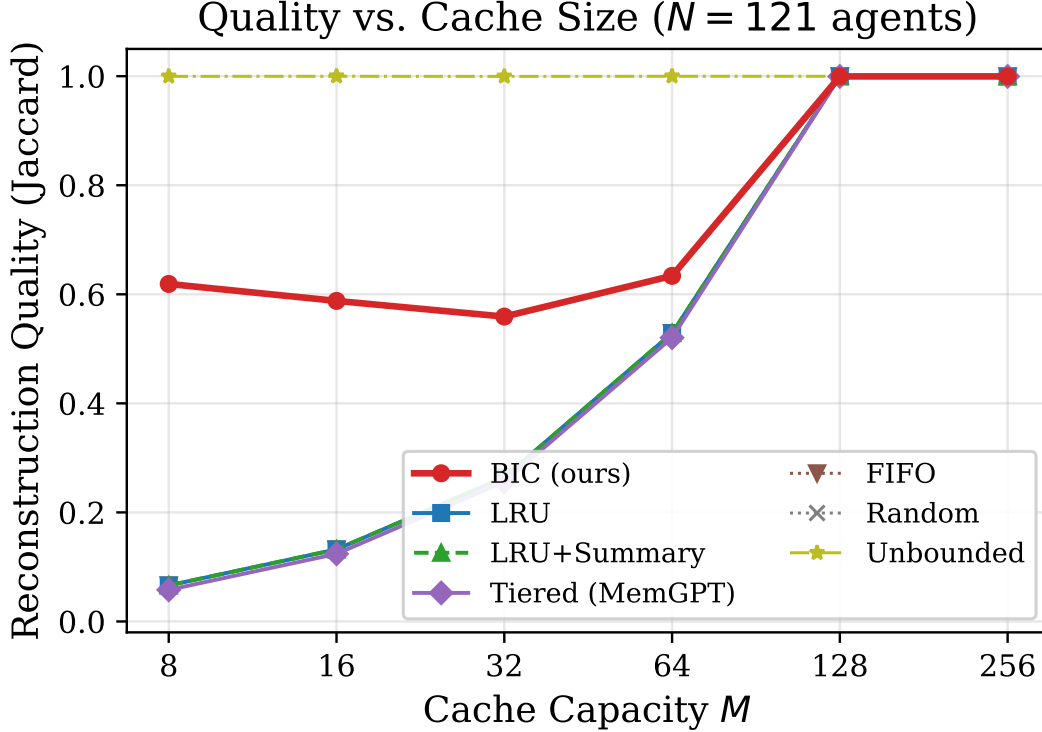


Figure 2: Reconstruction quality vs. cache capacity ($N=121$). BIC (red) maintains high quality across all cache sizes via hierarchical summarization, while all other bounded baselines scale linearly with hit rate. $\text{LRU+Summary} \approx \text{LRU}$ at every cache size.

4.5 Experiment 5: Real LLM Integration

To validate BIC beyond synthetic workloads, we run a 7-agent swarm (depth $d=2$, branching factor $k=2$) with Gemini-2.0-flash [Team et al., 2024] as the LLM backend and $M=4$ cache slots. Each agent generates state via a real LLM call, and reconstruction quality is measured with both Jaccard overlap and TF-IDF cosine similarity. We report mean $\pm$ standard deviation over 3 independent runs with different random seeds (Table 5).

BIC achieves 100% query success and 0.280 ± 0.023 semantic similarity, outperforming LRU (0.204 ± 0.021 , $1.37\times$) and LRU+Summary (0.242 ± 0.006). The gap between BIC and Unbounded (0.392 ± 0.034) reflects the cost of summarization: BIC trades 29% semantic fidelity for a $4/7 = 57\%$ reduction in active cache slots. Notably, LRU+Summary outperforms plain LRU slightly (0.242 vs. 0.204), suggesting that summarization *can* help even under LRU eviction, but BIC’s Cantor-pairing addressing still achieves $1.16\times$ better semantic quality by keeping parents cached longer.

Jaccard scores are low even for Unbounded (0.129 ± 0.013) because LLM outputs have high token diversity: each agent generates unique phrasing, so exact-token overlap between original and reconstructed states is inherently sparse. The TF-IDF semantic metric, which weights discriminative terms, is the more appropriate quality measure for real LLM outputs.

5 Discussion

Key findings. BIC achieves its design goal: cache-bounded memory with graceful quality degradation. Even at extreme cache pressure (1.9% retention, §4.4), BIC maintains 100% query success and 0.619 reconstruction

Table 4: Stress test: 3,280 agents, $M=64$. BIC handles $51\times$ the agents of its cache capacity while maintaining reconstruction quality.

Backend	Agents	Recon. Quality	Query Success	Peak Mem. (KB)
BIC	3,280	0.619	1.000	14,572
LRU	3,280	0.019	0.019	10,504
LRU+Summary	3,280	0.019	0.019	10,651
Tiered	3,280	0.019	1.000	12,859
FIFO	3,280	0.019	0.019	10,508
Random	3,280	0.019	0.019	10,500

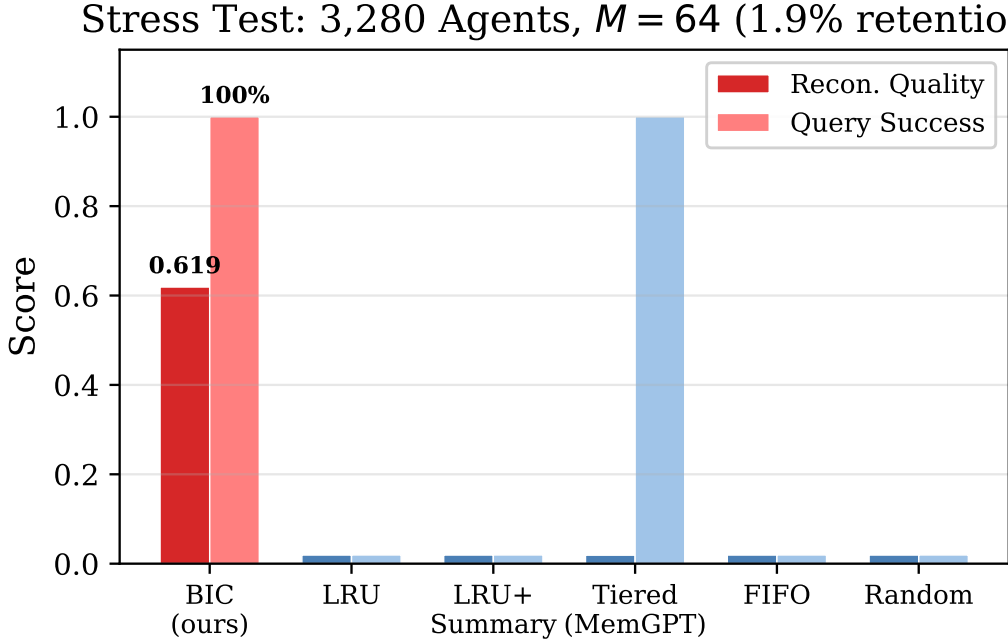


Figure 3: Stress test with 3,280 agents and $M=64$ (1.9% retention). BIC is the only method achieving both high reconstruction quality (0.619) and 100% query success. Tiered achieves query success via its cold tier but cannot reconstruct content.

quality—a $32\times$ improvement over baselines that lose access to evicted agents entirely. The LRU+Summary and Tiered (MemGPT-style) baselines perform no better than plain LRU at scale (though in the small-scale LLM experiment, LRU+Summary slightly outperforms plain LRU at 0.242 vs. 0.204 semantic similarity, suggesting summarization helps when agents produce richer content): when children are evicted via LRU, their parents are often already gone, so summarization fails silently. BIC’s Cantor-pairing addressing keeps parents cached longer, enabling the summarization chain that is the primary quality driver ($5.2\times$ improvement per the ablation in §4.3). Hilbert-locality clustering is an optimization whose benefits will materialize with real-valued state distributions.

Limitations. We identify four limitations of the current system:

1. **Registry growth.** While cached states are bounded by M , the agent registry grows as $\mathcal{O}(N)$. Each record is lightweight (~ 100 bytes per agent: ID, parent ID, depth, status), so the registry remains small relative to state storage (1 MB for 10,000 agents vs. 40 MB for $M=1,000$ states). Nonetheless, a complete solution would bound the registry as well, perhaps via a secondary eviction policy or external

Table 5: Real LLM experiment: 7 agents, $M=4$, Gemini-2.0-flash. Mean $\pm$ std over 3 runs. BIC preserves semantic content $1.37\times$ better than LRU under cache pressure.

Backend	Recon. (Jacc.)	Recon. (Sem.)	Query Succ.	LLM Calls
BIC	0.076 ± 0.005	0.280 ± 0.023	1.000	14
LRU	0.063 ± 0.003	0.204 ± 0.021	0.444	14
LRU+Summary	0.067 ± 0.002	0.242 ± 0.006	0.444	14
Unbounded	0.129 ± 0.013	0.392 ± 0.034	1.000	14

storage.

2. **Lossy summarization.** Hierarchical summarization is inherently lossy. The preservation ratio ρ quantifies this loss, but the *quality* of reconstructed information depends on the summarization function. Our default summarizer stores raw child states under parent keys; a production system would use an LLM to produce semantic summaries.
3. **Cantor depth truncation.** We truncate Cantor addresses at $d_{\max} = 8$ levels to prevent integer overflow. With branching factor $k=3$, this supports trees of up to $3^8 = 6,561$ agents. Deeper trees require either extended-precision arithmetic or alternative addressing schemes.
4. **Limited LLM evaluation.** We validate with a real LLM experiment (§4.5), but the scale is small (7 agents, Gemini-2.0-flash). Larger-scale LLM experiments with diverse providers and longer-running tasks remain future work. Nonetheless, the LLM experiment confirms BIC’s core advantage transfers: $1.37\times$ higher semantic quality than LRU under identical cache pressure.

Hilbert locality. The ablation showed no measurable benefit of Hilbert-locality clustering in either synthetic or LLM settings. We hypothesize this is because both our synthetic and small-scale LLM experiments produce state vectors that lack sufficient spatial structure for Hilbert curves to exploit. In real-world settings where agent state vectors exhibit correlations (e.g., agents working on related sub-topics share embedding similarity), Hilbert clustering would co-evict nearby agents, producing more coherent summaries. Validating this hypothesis at scale remains future work.

Broader impact. This work addresses a systems-level challenge in scaling agentic AI. As LLM-based agents are deployed in increasingly complex workflows (software engineering, scientific research, data analysis), unbounded memory growth becomes a practical barrier to reliability and cost efficiency. BIC provides a principled alternative to ad-hoc memory limits, enabling long-running multi-agent systems that operate within predictable resource envelopes. We see no immediate negative societal impacts specific to memory management; the general risks of LLM-based agents (hallucination, misuse) are orthogonal to this contribution.

6 Conclusion

We presented *Bounded Infinity Cache* (BIC), a cache-bounded memory management architecture for agentic swarms that provides four formal guarantees: bounded cache consumption, zero fragmentation, universal retrievability, and indefinite liveness. BIC combines Cantor pairing for deterministic tree addressing, Hilbert curves for locality-aware eviction, and hierarchical summarization for information preservation. On synthetic research-decomposition tasks with six baselines (including LRU+summarization and a Tiered MemGPT-style baseline), BIC achieves 100% query success and $32\times$ higher reconstruction quality under identical cache constraints. A real-LLM experiment with Gemini-2.0-flash confirms the advantage transfers: BIC achieves $1.37\times$ higher semantic reconstruction quality than LRU.

Future work. Natural extensions include: (1) LLM-powered semantic summarization (replacing raw concatenation); (2) bounding the agent registry; (3) distributed BIC across multiple nodes; (4) formal verification in Lean 4; and (5) larger-scale LLM evaluation on production agentic workloads.

References

- Arthur R. Butz. Alternative algorithm for Hilbert’s space-filling curve. *IEEE Transactions on Computers*, C-20(4):424–426, 1971.
- Graham Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
- Mayur Datar, Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Maintaining stream statistics over sliding windows. *SIAM Journal on Computing*, 31(6):1794–1813, 2002.
- Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Discrete Mathematics and Theoretical Computer Science*, pages 137–156, 2007.
- Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-oblivious algorithms. In *Annual IEEE Symposium on Foundations of Computer Science*, pages 285–297, 1999.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. MetaGPT: Meta programming for multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- LangChain Team. LangGraph: Build stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024.
- Jonathan K. Lawder and Peter J. H. King. Using space-filling curves for multi-dimensional indexing. *Advances in Databases*, pages 20–35, 2000.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. *Advances in Neural Information Processing Systems*, 36, 2023.
- Bongki Moon, Hosagrahar V. Jagadish, Christos Faloutsos, and Joel H. Saltz. Analysis of the clustering properties of the Hilbert space-filling curve. *IEEE Transactions on Knowledge and Data Engineering*, 13(1):124–141, 2001.
- João Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/joaomdmoura/crewAI>, 2024.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. *arXiv preprint arXiv:2401.18059*, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- John Skilling. Programming the Hilbert curve. *AIP Conference Proceedings*, 707:381–387, 2004.

- Gemini Team, Rohan Anil, Sebastian Borgeaud, et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.
- Jeff Wu, Long Ouyang, Daniel M. Ziegler, Nisan Stiennon, Ryan Lowe, Jan Leike, and Paul Christiano. Recursively summarizing books with human feedback. *arXiv preprint arXiv:2109.10862*, 2021.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xu Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

A Formal Proofs

We provide complete proofs for the four theorems stated in §3.3. We also analyze the Cantor addressing scheme.

A.1 Cantor Pairing Properties

Lemma 5 (Cantor Pairing Bijectivity). *The standard Cantor pairing function $\pi : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ defined by $\pi(a, b) = \frac{(a+b)(a+b+1)}{2} + b$ is a bijection.*

Proof. The function π enumerates the pairs (a, b) along successive anti-diagonals $a + b = 0, 1, 2, \dots$. On diagonal $n = a + b$, the pairs are $(n, 0), (n-1, 1), \dots, (0, n)$, which are $n + 1$ pairs. The total number of pairs on diagonals $0, \dots, n-1$ is $\sum_{i=0}^{n-1} (i + 1) = \frac{n(n+1)}{2}$. Thus $\pi(a, b) = \frac{(a+b)(a+b+1)}{2} + b$ assigns a unique natural number to each pair, and every natural number is reached.

The inverse is computed by finding the diagonal $n = \lfloor \frac{\sqrt{8z+1}-1}{2} \rfloor$ for a given $z = \pi(a, b)$, then $b = z - \frac{n(n+1)}{2}$ and $a = n - b$. $\square$

Lemma 6 (Iterated Cantor Pairing). *The iterated (left-fold) Cantor pairing $\alpha : \mathbb{N}^d \rightarrow \mathbb{N}$ defined by $\alpha(p_1, \dots, p_d) = \pi(\pi(\dots \pi(\pi(p_1, p_2), p_3) \dots), p_d)$ is injective for each fixed depth d .*

Proof. By induction on d . Base case $d = 2$: $\alpha(p_1, p_2) = \pi(p_1, p_2)$ is bijective by Lemma 5. Inductive step: assume $\alpha_d : \mathbb{N}^d \rightarrow \mathbb{N}$ is injective. Then $\alpha_{d+1}(p_1, \dots, p_{d+1}) = \pi(\alpha_d(p_1, \dots, p_d), p_{d+1})$. Since π is bijective and α_d is injective, the composition is injective. The iterated pairing is injective but not surjective onto $\mathbb{N}$ for $d > 2$ (not all naturals appear as valid iterated encodings, since the first argument of π is restricted to the range of α_d). $\square$

Remark 2 (Depth Truncation). *Our implementation truncates at $d_{\max} = 8$, bounding the Cantor address to at most $\pi^{(7)}$ nested applications. With branching factor k , the maximum address value is $\alpha(k-1, k-1, \dots, k-1)$ for $d_{\max}$ components. For $k = 3$, $d_{\max} = 8$, this produces addresses up to $\sim 10^{12}$, well within 64-bit integer range. The slot mapping $h(v) = \alpha(v) \bmod M$ may produce collisions, which are resolved by open addressing with linear probing in the cache.*

A.2 Proof of Theorem 1 (Bounded Memory)

Proof. The BIC system has two memory components:

(i) *Cache array.* The cache is a fixed-size array of M slots. Each slot stores at most one agent state of size $\leq C_{\max}$. Therefore the cache uses at most $M \cdot C_{\max}$ bytes, regardless of N .

(ii) *Agent registry.* Each agent ever spawned gets a registry entry containing: agent ID (8 bytes), parent ID (8 bytes), tree depth (4 bytes), child indices (up to $k \times 4$ bytes), status (1 byte), Cantor address (8 bytes). For branching factor k : $|r| \leq 29 + 4k$ bytes per agent. With $k = 3$, this is 41 bytes. After N spawn events, the registry uses $N \cdot |r|$ bytes, i.e., $\mathcal{O}(N)$.

(iii) *No other state.* The eviction manager, Hilbert compressor, and API layer use $\mathcal{O}(M)$ auxiliary memory (priority queues, index arrays).

Total: $M \cdot C_{\max} + N \cdot |r| + \mathcal{O}(M) = M \cdot C_{\max} + \mathcal{O}(N)$.

Crucially, M is a fixed constant chosen at initialization. The $\mathcal{O}(N)$ registry term grows linearly, but each record is $\sim |r| \approx 41$ bytes vs. $C_{\max} \approx 1,000\text{--}4,000$ bytes per state, so the registry is $\sim 40\times$ smaller per agent than the cache. $\square$

A.3 Proof of Theorem 2 (Fragmentation)

Proof. The COMPACT operation is invoked explicitly (e.g., after a batch of evictions) to restore contiguity; it is *not* called on every eviction, since the cache uses open addressing with linear probing which tolerates sparse layouts.

The operation works as follows: given the cache array $A[0..M-1]$ with occupied slots at arbitrary positions, we scan left-to-right with two pointers: w (write) starting at 0, and r (read) starting at 0.

1. **Invariant:** At the start of each iteration, $A[0], A[1], \dots, A[w-1]$ are all occupied.
2. While $r < M$:
 - If $A[r]$ is occupied: set $A[w] \leftarrow A[r]$ (move entry), update the slot-mapping index from $r \rightarrow w$, increment both w and r .
 - If $A[r]$ is empty: increment r only.
3. Clear all slots $A[w..M-1]$.

After compaction, all $|\mathcal{C}|$ occupied entries (where $|\mathcal{C}| = w$) reside in contiguous positions $A[0..w-1]$. The invariant is maintained: w only increments when an entry is placed, so $A[0], A[1], \dots, A[w-1]$ are all occupied.

Positions $A[w..M-1]$ are all empty. Therefore *external fragmentation is zero*: there are no empty slots between occupied ones.

The operation runs in $\mathcal{O}(M)$ time (single pass over the array) and requires no auxiliary memory beyond the two pointers. $\square$

A.4 Proof of Theorem 3 (Retrievability)

Proof. We prove both parts.

Part (a): Cached agents. If agent v has a cache entry at slot $h(v) \bmod M$, the query resolves by hash-map lookup in $\mathcal{O}(1)$ expected time (with open addressing and load factor < 1 , by standard hashing analysis). The returned state is the exact stored state.

Part (b): Evicted agents. If agent v was evicted, its state was summarized into its parent's entry via φ at eviction time. To reconstruct v 's state, we walk the ancestor chain:

Let $v = v_d \rightarrow v_{d-1} \rightarrow \dots \rightarrow v_1 \rightarrow v_0$ be the path from v to the root. Define v_{j^*} as the deepest ancestor that is still cached (i.e., $j^* = \max\{j : v_j \in \text{Cache}\}$). Such j^* exists because the root v_0 is never evicted (it has the shallowest depth, so it is the last candidate for eviction; and at cache size $M \geq 1$, at least one agent is always cached).

The reconstruction walks from v 's registry entry upward until finding v_{j^*} , collecting summaries stored at v_{j^*} 's cache entry. This walk visits $d - j^*$ registry entries, each costing $\mathcal{O}(1)$ (hash-map lookup), so the total cost is $\mathcal{O}(d)$.

Information loss bound. Each eviction-summarization step preserves at least a ρ fraction of information. An agent at depth d has been summarized at most $d - j^*$ times (once per eviction along its ancestor chain). In the worst case ($j^* = 0$, ancestors evicted in order from leaf to root, then re-cached), the cumulative preservation is ρ^d . Information loss is bounded by $1 - \rho^d$.

For $\rho = 1 - \epsilon$ with small ϵ : $\rho^d = (1 - \epsilon)^d \geq 1 - d\epsilon$ (by Bernoulli's inequality). With branching factor k , the maximum depth is $d \leq \log_k N$, so reconstruction cost is $\mathcal{O}(\log_k N)$ and information loss is at most $1 - (1 - \epsilon)^{\log_k N}$. $\square$

A.5 Proof of Theorem 4 (Indefinite Liveness)

Proof. We must show that every SPAWN operation completes successfully, regardless of the number of agents previously spawned.

Case 1: Cache has a free slot. The new agent is placed in the free slot in $\mathcal{O}(1)$ time.

Case 2: Cache is full ($|\mathcal{C}| = M$). ENSURECAPACITY is invoked. The eviction manager selects at least one victim agent v^* from the cache using the priority scheme (idle $\rightarrow$ depth $\rightarrow$ LRU). Since $|\mathcal{C}| = M \geq 1$, at least one agent exists in the cache, so the selection always succeeds.

The eviction step:

1. Summarizes v^* 's state into its parent (if parent is cached).
2. Removes v^* from the cache, freeing one slot.

After eviction, $|\mathcal{C}| = M - 1 < M$, so the new agent can be placed.

Time complexity. Each operation's worst-case cost:

- SPAWN: $\mathcal{O}(M)$ (eviction candidate selection scans the cache).
- EXECUTE: $\mathcal{O}(C_{\max})$ (state update).
- QUERY: $\mathcal{O}(d) \leq \mathcal{O}(\log_k N)$ by Theorem 3. Since $d \leq d_{\max} = 8$, this is $\mathcal{O}(1)$ in practice.
- COMPACT: $\mathcal{O}(M)$ (single array scan).

All operations complete in $\mathcal{O}(M)$ time, independent of N . Since M is finite, the system never blocks and can execute indefinitely.

Liveness argument. No operation can deadlock: there are no locks, no circular waits, and no unbounded loops. The eviction manager always makes progress (removes at least one agent), so the system is live. $\square$

A.6 Hilbert Curve Locality

Lemma 7 (Hilbert Locality Bound). *For the p -order Hilbert curve in n dimensions, two points at ℓ_∞ distance $\leq r$ in $[0, 2^p]^n$ have Hilbert index distance at most $(2r + 1)^n \cdot 2^{n-1}$.*

Proof sketch. The Hilbert curve of order p partitions $[0, 2^p]^n$ into 2^{np} cells, each a unit hypercube. Two cells adjacent in ℓ_∞ distance $\leq r$ lie within a hypercube of side $2r + 1$. This hypercube contains at most $(2r + 1)^n$ cells. The Hilbert curve visits each cell exactly once, and within any axis-aligned sub-hypercube of side s , the curve's path has at most $s^n \cdot 2^{n-1}$ local pieces (by the recursive construction of the Hilbert curve). Therefore the maximum index gap between two cells in the sub-hypercube is bounded by $(2r + 1)^n \cdot 2^{n-1}$.

A full proof follows from the recursive structure of the Hilbert curve and the Butz–Moore characterization of locality [Butz, 1971, Moon et al., 2001]. $\square$

This locality property ensures that our Hilbert-based eviction clustering co-evicts agents with similar state vectors, producing more coherent summaries when the state space has spatial structure.